

Striver's A2Z DSA Sheet: Comprehensive Topic Guide & Links

Contents

1	Step 01 - Basic	3
1.1	C++	3
1.2	Hashing	3
1.3	Maths	3
1.4	Pattern-Problems	3
1.5	Recursion - Basic Recursion Problems	3
1.6	Recursion - Basic Intro Rec, Rec Tree, Stack Space	3
1.7	STL	3
2	Step 02 - Sorting Techniques	3
2.1	Lec - 01	3
2.2	Lec - 02	3
3	Step 03 - Solve Problems On Arrays	3
3.1	Easy	3
3.2	Medium	4
3.3	Hard	4
4	Step 04 - Binary Search [1D, 2D Arrays, Search Space]	5
4.1	Lec 1 - BS on 1D Arrays	5
4.2	Lec 2 - BS on Answer	5
4.3	Lec 3 - BS on 2D Arrays	6
5	Step 06 - Learn LinkedList [Single LL, Double LL, Medium, Hard Problems]	6
5.1	Lec 1 - Learn 1D LinkedList	6
5.2	Lec 2 - Learn Doubly LinkedList	6
5.3	Lec 3 - Medium Problems of LL	6
5.4	Lec 4 - Medium Problems of DLL	7
5.5	Lec 5 - Hard Problems of LL	7
6	Step 07 - Recursion [PatternWise]	7
6.1	Lec 1 - Get a Strong Hold	7
6.2	Lec 2 - Subsequences Pattern	7
6.3	Lec 3 - Trying out all Combos Hard	8
7	Step 08 - Bit Manipulation [Concepts & Problems]	8
7.1	Lec 1 - Learn Bit Manipulation	8
7.2	Lec 2 - Interview Problems	8
7.3	Lec 3 - Advanced Maths	9

8	Step 09 - Stack and Queues [Learning, Pre-In-Post-fix, Monotonic Stack, Implementation]	9
8.1	Lec 1 - Learning	9
8.2	Lec 2 - Prefix, Infix, PostFix Conversion Problems	9
8.3	Lec 3 - Monotonic StackQueue Problems [VVV. Imp]	9
8.4	Lec 4 - Implementation Problems	10
9	Step 10 - Sliding Window & Two Pointer Combined Problems	10
9.1	Lec 1 - Medium Problems	10
9.2	Lec 2 - Hard Problems	10
10	Step 12 - Greedy Algorithms [Easy, MediumHard]	11
10.1	Lec 1 - Easy Problems	11
10.2	Lec 2 - Medium & Hard	11
11	Step 13 - Binary Trees [Traversals, Medium and Hard Problems]	11
11.1	Lec 1 - Traversals	11
11.2	Lec 2 - Medium Problems	12
11.3	Lec 3 - Hard Problems	12

1 Step 01 - Basic

1.1 C++

- `c++.cpp` [\[File\]](#)

1.2 Hashing

- `hash.cpp` [\[File\]](#)

1.3 Maths

- `maths.cpp` [\[File\]](#)

1.4 Pattern-Problems

- `patterns.cpp` [\[File\]](#)

1.5 Recursion - Basic Recursion Problems

- `recursion-prob.cpp` [\[File\]](#)

1.6 Recursion - Basic Intro Rec, Rec Tree, Stack Space

- `recursion.cpp` [\[File\]](#)

1.7 STL

- `main.cpp` [\[File\]](#)

2 Step 02 - Sorting Techniques

2.1 Lec - 01

- `sorting.cpp` [\[File\]](#)

2.2 Lec - 02

- `sorting.cpp` [\[File\]](#)

3 Step 03 - Solve Problems On Arrays

3.1 Easy

- `finding-intersection.cpp` [\[LeetCode\]](#)
- `finding-union.cpp` [\[GFG\]](#)
- `isSorted.cpp` [\[LeetCode\]](#)
- `largest-element-in-array.cpp` [\[GFG\]](#)

- `left-rotate-by-one.cpp` [\[LeetCode\]](#)
- `linear-search.cpp` [\[GFG\]](#)
- `longest-subarray-with-sum-k-pos-neg.cpp` [\[LeetCode\]](#)
- `longest-subarray-with-sum-k.cpp` [\[GFG\]](#)
- `maximum-consecutive-ones.cpp` [\[LeetCode\]](#)
- `missing-number.cpp` [\[LeetCode\]](#)
- `move-zero.cpp` [\[LeetCode\]](#)
- `remove-duplicate-from-sorted-array.cpp` [\[LeetCode\]](#)
- `second-largest-element.cpp` [\[GFG\]](#)
- `singleNumber.cpp` [\[LeetCode\]](#)

3.2 Medium

- `2sum.cpp` [\[LeetCode\]](#)
- `count-sub-array-with-given-sum.cpp` [\[LeetCode\]](#)
- `leaders-in-array-prob.cpp` [\[GFG\]](#)
- `longest-consecutive-sequence.cpp` [\[LeetCode\]](#)
- `majority-element.cpp` [\[LeetCode\]](#)
- `maximum-subarray.cpp` [\[LeetCode\]](#)
- `next-permutation.cpp` [\[LeetCode\]](#)
- `rearrange-array-by-sign.cpp` [\[LeetCode\]](#)
- `rotate-matrix-by-90deg.cpp` [\[LeetCode\]](#)
- `set-matrix-zero.cpp` [\[LeetCode\]](#)
- `sortArrayOf0-1-2.cpp` [\[LeetCode\]](#)
- `spiral-matrix.cpp` [\[LeetCode\]](#)
- `stock-buy-sell.cpp` [\[LeetCode\]](#)

3.3 Hard

- `3sum.cpp` [\[LeetCode\]](#)
- `4Sum.cpp` [\[LeetCode\]](#)
- `count-inversion.cpp` [\[GFG\]](#)
- `find-missing-and-repeating-number.cpp` [\[LeetCode\]](#)
- `largest-subarray-with-sum-zero.cpp` [\[GFG\]](#)
- `majority-element.cpp` [\[LeetCode\]](#)

- `maximum-product-subarray.cpp` [\[LeetCode\]](#)
- `merge-overlapping-subintervals.cpp` [\[LeetCode\]](#)
- `merge-two-sorted-array-without-extra-space.cpp` [\[LeetCode\]](#)
- `number-of-subarray-with-xor-zero.cpp` [\[GFG\]](#)
- `pascal-triangle.cpp` [\[LeetCode\]](#)
- `reverse-pairs.cpp` [\[LeetCode\]](#)

4 Step 04 - Binary Search [1D, 2D Arrays, Search Space]

4.1 Lec 1 - BS on 1D Arrays

- `bs-to-find-x-in-sorted-array.cpp` [\[LeetCode\]](#)
- `count-occurrence-in-array.cpp` [\[GFG\]](#)
- `find-how-many-time-has-array-been-rotated.cpp` [\[GFG\]](#)
- `find-peak-element.cpp` [\[LeetCode\]](#)
- `find-the-first-and-last-occurrence-in-sorted-array.cpp` [\[LeetCode\]](#)
- `floor-ceil-in-sorted-array.cpp` [\[GFG\]](#)
- `lower-bound.cpp` [\[GFG\]](#)
- `minimum-in-rotated-sorted-array-1.cpp` [\[LeetCode\]](#)
- `minimum-in-rotated-sorted-array-2.cpp` [\[LeetCode\]](#)
- `search-in-rotated-sorted-array-1.cpp` [\[LeetCode\]](#)
- `search-in-rotated-sorted-array-2.cpp` [\[LeetCode\]](#)
- `search-insert-position.cpp` [\[LeetCode\]](#)
- `single-element-in-sorted-array.cpp` [\[LeetCode\]](#)
- `upper-bound.cpp` [\[GFG\]](#)

4.2 Lec 2 - BS on Answer

- `aggressive-cows.cpp` [\[GFG\]](#)
- `allocate-books.cpp` [\[GFG\]](#)
- `capacity-to-ship-package-within-D-days.cpp` [\[LeetCode\]](#)
- `find-nth-root-of-m.cpp` [\[GFG\]](#)
- `find-sqrt-of-integer.cpp` [\[LeetCode\]](#)
- `find-the-smallest-divisor.cpp` [\[LeetCode\]](#)
- `koto-eating-banana.cpp` [\[LeetCode\]](#)

- `kth-element-of-two-sorted-array.cpp` [\[GFG\]](#)
- `kth-missing-positive-number.cpp` [\[LeetCode\]](#)
- `median-of-two-sorted-array.cpp` [\[LeetCode\]](#)
- `min-n0-of-days-to-make-m-bouquets.cpp` [\[LeetCode\]](#)
- `minimise-maximum-distance-between-gas-station.cpp` [\[LeetCode\]](#)
- `painters-partition-and-split-array-largest-sum.cpp` [\[LeetCode\]](#)

4.3 Lec 3 - BS on 2D Arrays

- `median-in-a-row-wise-sorted-matrix.cpp` [\[GFG\]](#)
- `peak-element-in-2D-mat-p-2.cpp` [\[LeetCode\]](#)
- `row-with-maximum-ones.cpp` [\[LeetCode\]](#)
- `search-a-2D-matrix-p-2.cpp` [\[LeetCode\]](#)
- `search-a-2D-matrix.cpp` [\[LeetCode\]](#)

5 Step 06 - Learn LinkedList [Single LL, Double LL, Medium, Hard Problems]

5.1 Lec 1 - Learn 1D LinkedList

- `basic/LL.cpp` [\[File\]](#)
- `conver-array-to-LL.cpp` [\[File\]](#)
- `delete-node-in-a-linkedlist.cpp` [\[LeetCode\]](#)
- `deletion-in-LL.cpp` [\[File\]](#)
- `insertion-in-LL.cpp` [\[File\]](#)
- `length-of-LL.cpp` [\[GFG\]](#)
- `search-in-LL.cpp` [\[GFG\]](#)

5.2 Lec 2 - Learn Doubly LinkedList

- `DLL.cpp` [\[File\]](#)
- `reverse-doubly-linkedlist.cpp` [\[GFG\]](#)

5.3 Lec 3 - Medium Problems of LL

- `add-1-to-a-linkedlist.cpp` [\[GFG\]](#)
- `add-two-numbers-in-linkedlist.cpp` [\[LeetCode\]](#)
- `delete-the-middle-node-of-linkedlist.cpp` [\[LeetCode\]](#)

- `detect-loop-in-LL.cpp` [\[LeetCode\]](#)
- `find-length-of-loop.cpp` [\[GFG\]](#)
- `insertion-of-two-LL.cpp` [\[LeetCode\]](#)
- `middle-of-the-linkedlist.cpp` [\[LeetCode\]](#)
- `odd-even-linkedlist.cpp` [\[LeetCode\]](#)
- `palindrome-linked-list.cpp` [\[LeetCode\]](#)
- `remove-nth-node-from-end-of-list.cpp` [\[LeetCode\]](#)
- `reverse-linkedlist.cpp` [\[LeetCode\]](#)
- `sort-linked-list-0s-1s-2s.cpp` [\[GFG\]](#)
- `sort-list.cpp` [\[LeetCode\]](#)
- `starting-node-in-a-loop-of-LL.cpp` [\[LeetCode\]](#)

5.4 Lec 4 - Medium Problems of DLL

- `delete-all-occurrence-of-given-key-in-DLL.cpp` [\[GFG\]](#)
- `find-pais-of-sum-in-DLL.cpp` [\[GFG\]](#)
- `remove-duplicates-from-sorted-DLL.cpp` [\[GFG\]](#)

5.5 Lec 5 - Hard Problems of LL

- `copy-linkedlist-with-random-pointer.cpp` [\[LeetCode\]](#)
- `flatten-linkedlist.cpp` [\[GFG\]](#)
- `reverse-nodes-in-kth-group.cpp` [\[LeetCode\]](#)
- `rotate-LL.cpp` [\[LeetCode\]](#)

6 Step 07 - Recursion [PatternWise]

6.1 Lec 1 - Get a Strong Hold

- `pow-x-n.cpp` [\[LeetCode\]](#)

6.2 Lec 2 - Subsequences Pattern

- `combination-sum-II.cpp` [\[LeetCode\]](#)
- `combination-sum.cpp` [\[LeetCode\]](#)
- `count-the-subsequence-with-sum-k.cpp` [\[GFG\]](#)
- `print-any-one-subsequence-whose-sum-is-k.cpp` [\[File\]](#)
- `print-subsequence-whose-sum-is-k.cpp` [\[File\]](#)

- `subset(power-set).cpp` [\[LeetCode\]](#)
- `subset-sum-II.cpp` [\[LeetCode\]](#)
- `subset-sum.cpp` [\[GFG\]](#)

6.3 Lec 3 - Trying out all Combos Hard

- `M-Coloring-Problems.cpp` [\[GFG\]](#)
- `N-Queen.cpp` [\[LeetCode\]](#)
- `palindrome-partitioning.cpp` [\[LeetCode\]](#)
- `rat-in-maze.cpp` [\[GFG\]](#)
- `sudoku-solver.cpp` [\[LeetCode\]](#)

7 Step 08 - Bit Manipulation [Concepts & Problems]

7.1 Lec 1 - Learn Bit Manipulation

- `divide-two-integers.cpp` [\[LeetCode\]](#)
- `kth-bit-set-or-not.cpp` [\[GFG\]](#)
- `number-odd-even.cpp` [\[GFG\]](#)
- `one-liner-code.cpp` [\[File\]](#)
- `power-of-two.cpp` [\[LeetCode\]](#)
- `set-ith-bit.cpp` [\[GFG\]](#)
- `set-unset-the-rightmost-bit.cpp` [\[File\]](#)
- `swap-two-numbers.cpp` [\[GFG\]](#)

7.2 Lec 2 - Interview Problems

- `find-xor-of-num-from-L-to-R.cpp` [\[GFG\]](#)
- `min-bit-flips-to-convert-number.cpp` [\[LeetCode\]](#)
- `single-number-II.cpp` [\[LeetCode\]](#)
- `single-number-III.cpp` [\[LeetCode\]](#)
- `single-number.cpp` [\[LeetCode\]](#)
- `subsets.cpp` [\[LeetCode\]](#)

7.3 Lec 3 - Advanced Maths

- `all-divisor-number.cpp` [\[GFG\]](#)
- `power-exponentiation.cpp` [\[LeetCode\]](#)
- `prime-factorization-using-sieve.cpp` [\[GFG\]](#)
- `prime-number.cpp` [\[LeetCode\]](#)
- `print-prime-factor-of-a-number.cpp` [\[GFG\]](#)

8 Step 09 - Stack and Queues [Learning, Pre-In-Post-fix, Monotonic Stack, Implementation]

8.1 Lec 1 - Learning

- `implement-min-stack.cpp` [\[LeetCode\]](#)
- `implement-queue-using-LL.cpp` [\[GFG\]](#)
- `implement-queue-using-array.cpp` [\[GFG\]](#)
- `implement-stack-using-LL.cpp` [\[GFG\]](#)
- `implement-stack-using-array.cpp` [\[GFG\]](#)
- `implement-stack-using-queue.cpp` [\[LeetCode\]](#)
- `valid-parentheses.cpp` [\[LeetCode\]](#)

8.2 Lec 2 - Prefix, Infix, PostFix Conversion Problems

- `infix-to-postfix.cpp` [\[GFG\]](#)
- `infix-to-prefix.cpp` [\[File\]](#)
- `one-liner-code.cpp` [\[File\]](#)
- `postfix-to-infix.cpp` [\[GFG\]](#)
- `postfix-to-prefix.cpp` [\[GFG\]](#)
- `prefix-to-infix.cpp` [\[GFG\]](#)
- `prefix-to-postfix.cpp` [\[GFG\]](#)

8.3 Lec 3 - Monotonic StackQueue Problems [VVV. Imp]

- `asteroid-collison.cpp` [\[LeetCode\]](#)
- `largest-rectangle-in-histogram.cpp` [\[LeetCode\]](#)
- `maximal-rectangle.cpp` [\[LeetCode\]](#)
- `nearest-smallest-element.cpp` [\[File\]](#)
- `next-greater-element-II.cpp` [\[LeetCode\]](#)

- `next-greater-element.cpp` [\[LeetCode\]](#)
- `remove-k-digits.cpp` [\[LeetCode\]](#)
- `sum-of-subarray-minimum.cpp` [\[LeetCode\]](#)
- `sum-of-subarray-ranges.cpp` [\[LeetCode\]](#)
- `trapping-rain-water.cpp` [\[LeetCode\]](#)

8.4 Lec 4 - Implementation Problems

- `lfu-cache.cpp` [\[LeetCode\]](#)
- `lru-cache(imp).cpp` [\[LeetCode\]](#)
- `online-stock-span.cpp` [\[LeetCode\]](#)
- `sliding-window-maximum.cpp` [\[LeetCode\]](#)
- `the-celebrity-problem.cpp` [\[LeetCode\]](#)

9 Step 10 - Sliding Window & Two Pointer Combined Problems

9.1 Lec 1 - Medium Problems

- `binary-subarrays-with-sum.cpp` [\[LeetCode\]](#)
- `count-number-of-nice-subarrays.cpp` [\[LeetCode\]](#)
- `fruit-into-baskets.cpp` [\[LeetCode\]](#)
- `intro.cpp` [\[File\]](#)
- `longest-repeating-character-replacement.cpp` [\[LeetCode\]](#)
- `longest-substring-without-repeating-characters.cpp` [\[LeetCode\]](#)
- `max-consecutive-ones-iii.cpp` [\[LeetCode\]](#)
- `maximum-points-you-can-obtain-from-cards.cpp` [\[LeetCode\]](#)
- `number-of-substrings-containing-all-three-characters.cpp` [\[LeetCode\]](#)

9.2 Lec 2 - Hard Problems

- `longest-substring-with-at-most-k-distinct-characters.cpp` [\[LeetCode\]](#)
- `minimum-window-substring.cpp` [\[LeetCode\]](#)
- `subarrays-with-k-different-integers.cpp` [\[LeetCode\]](#)

10 Step 12 - Greedy Algorithms [Easy, MediumHard]

10.1 Lec 1 - Easy Problems

- `assign-cookies.cpp` [\[LeetCode\]](#)
- `find-minimum-number-of-coins.cpp` [\[GFG\]](#)
- `fractional-knapsack.cpp` [\[GFG\]](#)
- `lemonade-change.cpp` [\[LeetCode\]](#)
- `valid-parenthesis-string.cpp` [\[LeetCode\]](#)

10.2 Lec 2 - Medium & Hard

- `candy.cpp` [\[LeetCode\]](#)
- `insert-interval.cpp` [\[LeetCode\]](#)
- `job-sequencing-problem.cpp` [\[GFG\]](#)
- `jump-game-II.cpp` [\[LeetCode\]](#)
- `jump-game-i.cpp` [\[LeetCode\]](#)
- `merge-intervals.cpp` [\[LeetCode\]](#)
- `minimum-platforms.cpp` [\[GFG\]](#)
- `n-meetings-in-one-room.cpp` [\[GFG\]](#)
- `non-overlapping-intervals.cpp` [\[LeetCode\]](#)
- `shortest-job-first.cpp` [\[GFG\]](#)

11 Step 13 - Binary Trees [Traversals, Medium and Hard Problems]

11.1 Lec 1 - Traversals

- `binary-tree-inorder-traversal.cpp` [\[LeetCode\]](#)
- `binary-tree-level-order-traversal.cpp` [\[LeetCode\]](#)
- `binary-tree-postorder-traversal.cpp` [\[LeetCode\]](#)
- `binary-tree-preorder-traversal.cpp` [\[LeetCode\]](#)
- `iterative-inorder-traversal-of-binary-tree.cpp` [\[File\]](#)
- `iterative-preorder-traversal-of-binary-tree.cpp` [\[File\]](#)
- `post-order-traversal-of-binary-tree-using-1-stack.cpp` [\[File\]](#)
- `post-order-traversal-of-binary-tree-using-2-stack.cpp` [\[File\]](#)
- `pre-post-in-order-in-one-traversal.cpp` [\[File\]](#)

11.2 Lec 2 - Medium Problems

- `balanced-binary-tree.cpp`
- `binary-tree-maximum-path-sum.cpp`
- `diameter-of-binary-tree.cpp`
- `maximum-depth-of-binary-tree.cpp`

[\[LeetCode\]](#)

[\[LeetCode\]](#)

[\[LeetCode\]](#)

[\[LeetCode\]](#)

11.3 Lec 3 - Hard Problems

- `maximum-width-of-binary-tree.cpp`

[\[LeetCode\]](#)